Post-contenido — Manejo de pantalla y teclado

Arquitectura de Computadores · Unidad 7

Estudiante: González López

Repositorio: TrabajosArqui — post7-u7

PDF: Gonzalez Lopez_Post7_U7.pdf

URL exacta del proyecto:
`https://github.com/alejandro1152113/TrabajosArqui/tree/main/unidad-7/post7-u7`

Programa fuente de ejemplo: `src/post1.asm` — `https://github.com/alejandro1152113/TrabajosArqui/blob/main/unidad-7/post7-u7/src/post1.asm`

Estado de validación: los nueve programas se ensamblaron correctamente con NASM 2.16.03 y sus `.COM` están en `bin/`. DOSBox 0.74-3 está instalado; las capturas y la comprobación visual de las salidas siguen pendientes. No se presentan ejecuciones ni evidencias ficticias.

Contenido del repositorio

- `src/`: nueve fuentes ensamblador de 16 bits.

- `build.bat`: compila los nueve programas con NASM instalado en el PATH de Windows.

- `bin/`: nueve archivos `.COM` generados desde los fuentes NASM.

- `capturas/`: lista de evidencias que deben obtenerse en DOSBox.

- PDF de esta documentación enlazado al inicio.

Entorno y compilación

Los programas son archivos `.COM` de 16 bits en modo real. Cada fuente usa `BITS 16`, `ORG 100h`, registros de 16 bits y etiquetas de datos dentro del mismo segmento; no usa directivas de objeto ELF ni segmentos `.data`/`.text` de un ejecutable protegido.

Requisitos de ejecución: DOSBox 0.74 o superior y NASM accesible desde DOSBox. El NASM instalado en Windows puede ensamblar los archivos desde el host, pero no sustituye al ejecutable DOS que la guía del curso pide verificar dentro de DOSBox.

Desde PowerShell, en esta carpeta, se puede recompilar en Windows con:

```
.\build.bat
```

Para compilar dentro de DOSBox, montar esta carpeta como unidad C, entrar en `C:\` y ejecutar:

```
nasm -f bin src\post1.asm  -o bin\post1.com
nasm -f bin src\post1b.asm -o bin\post1b.com
nasm -f bin src\post1c.asm -o bin\post1c.com
nasm -f bin src\post1d.asm -o bin\post1d.com
nasm -f bin src\post2a.asm -o bin\post2a.com
nasm -f bin src\post2b.asm -o bin\post2b.com
nasm -f bin src\post2c.asm -o bin\post2c.com
nasm -f bin src\post2d.asm -o bin\post2d.com
nasm -f bin src\post2e.asm -o bin\post2e.com
```

La compilación de esta entrega se realizó con NASM 2.16.03 para Win64. Para ejecutar y verificar cada archivo `bin\post*.com`, montar la carpeta en DOSBox y adaptar la ruta al lugar donde esté el repositorio:

```
mount c "C:\ruta\al\repositorio\unidad-7\post7-u7"
c:
```

Parte 1 — INT 21h e INT 10h

Programa | Interrupciones y propósito | Resultado esperado

`post1.asm` | INT 21h/AH=09h imprime cadenas $-terminadas; AH=4Ch finaliza el proceso. | Muestra las tres líneas del encabezado.

`post1b.asm` | INT 10h/AH=06h limpia la pantalla; AH=02h posiciona el cursor; AH=09h escribe un carácter y su atributo. | Muestra A amarillo sobre azul y U7 rojo claro sobre negro.

`post1c.asm` | Recorre una cadena y combina INT 10h/AH=02h con AH=09h para escribir carácter a carácter. | Muestra el título en verde brillante desde la fila 5, columna 25.

`post1d.asm` | INT 10h/AH=0Fh consulta modo, columnas y página; AH=08h relee carácter y atributo. | Informa si modo 03h y 80 columnas están activos, escribe Z y confirma la relectura de Z/2Fh.

INT 21h/AH=09h escribe a la salida estándar y, en DOS 2+, esta puede estar redirigida; la cadena termina en $. INT 10h/AH=09h es otra función distinta: escribe un carácter con atributo en la página seleccionada y no avanza el cursor. INT 10h/AH=02h coloca el cursor; por eso la rutina de cadena lo reposiciona para cada carácter. INT 10h/AH=0Fh devuelve el modo actual en AL, las columnas en AH y la página activa en BH. INT 10h/AH=08h devuelve el carácter en AL y el atributo en AH en la posición actual del cursor.

Parte 2 — INT 16h y acceso directo a B800h

Programa | Técnica | Resultado esperado

`post2a.asm` | INT 16h/AH=00h espera una tecla y entrega scan code en AH y ASCII en AL; la salida hexadecimal usa INT 21h/AH=02h. | Presenta ambos códigos por tecla; Escape termina.

`post2b.asm` | Escribe pares carácter/atributo directamente en ES:B800h usando un offset calculado. | Muestra H, O, L y A en cuatro filas y colores distintos.

`post2c.asm` | REP STOSW llena 2000 celdas; después escribe una cadena en el buffer de video. | Limpia el modo texto con fondo azul y muestra un mensaje en la fila 12.

`post2d.asm` | Editor de una línea con INT 16h/AH=00h y escritura directa en B800h. | Escribe caracteres imprimibles en la fila 12; Backspace borra y Enter termina.

`post2e.asm` | INT 16h/AH=02h consulta los flags; AH=01h comprueba si hay una tecla sin bloquear. | Los indicadores Shift/Ctrl se actualizan en pantalla; Escape termina.

En modo texto color 80×25, cada celda ocupa dos bytes: ASCII y atributo. Para fila `r` y columna `c`, el offset en bytes es `(r × 80 + c) × 2`; el carácter está en `offset` y el atributo en `offset+1`. Por ejemplo, fila 12/columna 40 produce `07D0h`. El atributo `0Ah` representa texto verde brillante sobre fondo negro; `1Eh`, texto amarillo sobre fondo azul; y `2Fh`, texto blanco brillante sobre fondo verde. En el byte de atributo, los cuatro bits bajos determinan el color del texto y los bits 4–6 el fondo; el bit 7 puede representar parpadeo o intensidad de fondo según la configuración.

INT 16h/AH=02h devuelve en AL el byte de estado mantenido por el BIOS. Los bits 0 y 1 representan Shift derecho e izquierdo; el bit 2 representa Ctrl. El programa `post2e.asm` sondea este estado sin bloquear, y usa AH=01h/AH=00h únicamente para detectar y consumir Escape.

Decisiones técnicas

1. ¿INT 21h o INT 10h para una salida de texto?

Para texto plano que solo debe aparecer en la salida estándar, `post1.asm` usa INT 21h/AH=09h: DOS procesa una cadena completa y la escritura puede seguir la redirección de STDOUT disponible desde DOS 2. Esa función no recibe un atributo de color ni una coordenada, por lo que no basta para los ejemplos que deben pintar caracteres en ubicaciones exactas. En `post1b.asm` y `post1c.asm`, INT 10h/AH=02h y AH=09h son necesarios para controlar la posición y el atributo de video. La ruta BIOS requiere más instrucciones cuando se recorre carácter por carácter, pero ofrece control directo de la presentación. El programa no debe confundir las dos funciones AH=09h: la de INT 21h imprime una cadena $-terminada; la de INT 10h escribe un carácter con atributo y cantidad. La selección depende del contrato requerido: redirección y texto simple favorecen DOS; posición y color favorecen BIOS.

2. ¿Por qué consultar AH=0Fh y releer con AH=08h?

Asumir el modo 03h puede producir resultados erróneos si el programa se ejecuta con otro modo activo: las dimensiones, las páginas disponibles y el significado de los atributos pueden cambiar. INT 10h/AH=0Fh permite consultar el modo, el ancho de fila y la página activa antes de usar la convención esperada. `post1d.asm` informa si no encuentra modo 03h con 80 columnas, pero continúa para que la relectura también pueda observarse; el adaptador y la página de prueba siguen siendo una precondición del ejemplo. Después de escribir `Z` con atributo `2Fh`, AH=08h permite comprobar los bytes que el BIOS devuelve para la celda del cursor. Esta lectura detecta una discrepancia antes de que el programa dé por válida la actualización. Un editor de pantalla o una rutina de interfaz puede aplicar esa verificación al mantener sincronizado su modelo interno con el contenido visible. Frente a eso, leer B800h directamente es rápido, pero ata el código a un modo y formato concretos.

3. ¿INT 16h bloqueante o lectura directa del puerto 60h?

INT 16h/AH=00h es apropiada cuando el programa espera una tecla y puede detenerse hasta que llegue; así funciona `post2a.asm` y así recibe el editor su siguiente carácter. En cambio, `post2e.asm` debe actualizar continuamente los indicadores de Shift y Ctrl aunque no haya pulsaciones. Si usara AH=00h en el bucle, se bloquearía hasta la siguiente tecla y dejaría de reflejar cambios del estado durante esa espera. Leer el puerto 60h directamente puede evitar parte del trabajo del BIOS y entregar acceso de bajo nivel al controlador, pero obliga a gestionar el ciclo completo del controlador y las interrupciones, interpretar make/break, tratar teclas extendidas y considerar el conjunto de scan codes. Una lectura aislada del puerto no equivale a una interfaz de teclado completa. Para este laboratorio, INT 16h ofrece la abstracción correcta y mantiene el ejemplo compatible con DOSBox. El acceso directo es pertinente al desarrollar un sistema operativo o un controlador propio que no delegue el teclado en el BIOS.

4. ¿AH=02h o seguimiento manual de make/break?

Para mostrar el estado actual de Shift y Ctrl, INT 16h/AH=02h es más simple que mantener variables propias actualizadas a partir de cada evento. El BIOS conserva los bits de estado al procesar las interrupciones del teclado, y devuelve juntos Shift izquierdo/derecho y Ctrl. Un seguimiento manual debe reconocer cada make y su break, además de gestionar códigos extendidos y el orden de eventos. Si el programa no consume a tiempo el código de liberación, puede conservar un estado obsoleto y mostrar una tecla modificadora como si siguiera presionada. Consultar AH=02h evita que la aplicación sea responsable de reconstruir ese estado a partir de eventos perdidos; también permite distinguir Shift izquierdo del derecho mediante los bits 1 y 0. El seguimiento manual conviene cuando se necesitan eventos de transición y su instante exacto, o cuando el programa controla por sí mismo el manejador de teclado. Para el objetivo del laboratorio —reflejar el estado actual— AH=02h es la elección más directa.

Validación y evidencias

Los nueve fuentes deben ensamblarse con NASM sin errores y ejecutarse individualmente en DOSBox. La guía exige capturas de los checkpoints de compilación y ejecución; deben agregarse a `capturas/` después de realizarlos. En esta entrega no se afirma que se haya comprobado el funcionamiento en DOSBox, porque esas herramientas no están instaladas en este entorno.

Referencias técnicas

- Brown, R. (2000). *Ralf Brown's Interrupt List* (Release 61). https://fd.lod.bz/rbil/

- IBM. (1981). *IBM Personal Computer technical reference*. IBM.

- IBM. (1987). *Video Graphics Array technical reference*. IBM.

- Microsoft. (1986). *MS-DOS programmer's reference*. Microsoft Press.

- Schulman, A., Brown, R., Maxey, D., Michels, R. J., & Kyle, J. C. (1990). *Undocumented DOS*. Addison-Wesley.